

OpenText™ Identity Manager

Staging Guide

Version : 25.4

PDF Generated on : April 21, 2026

Table of Contents

1. Staging Guide	1
1.1. Staging OpenText Identity Manager Projects	2
1.1.1. Understanding Staging	3
1.1.2. Scope of Staging Documentation	5
1.1.3. Staging Use Cases	6
1.1.4. Using Designer for Staging OpenText Identity Manager Projects	7
1.1.5. Moving Authorizations During Staging	8
1.2. Preparing for Staging	9
1.2.1. Understanding Packages	10
1.2.2. Converting Configuration Files to Packages	11
1.2.3. Prerequisites	12
1.2.4. Identity Vault Structure	13
1.2.5. Driver Configuration	14
1.2.6. OpenText Identity Manager Objects	15
1.2.7. Rights	26
1.3. Staging OpenText Identity Manager Projects	28
1.3.1. Staging a Project for the First Time	29
1.3.2. Staging Changes in an Existing Project	35
1.3.3. Copying Java Class .jar Files Between Stages	37
1.3.4. Post-Staging Tasks	38
1.3.5. Changing the LDAP Properties	40
1.4. Staging Best Practices	41

1. Staging Guide

This *guide* provides step-by-step procedures to move your Identity Management solutions from one stage to the next. It is intended for individuals responsible for comprehending administration concepts and implementing a secure, distributed administration model.

1.1. Staging OpenText Identity Manager Projects

The information covered in the following sections helps you understand the basic principles of staging OpenText Identity Manager projects:

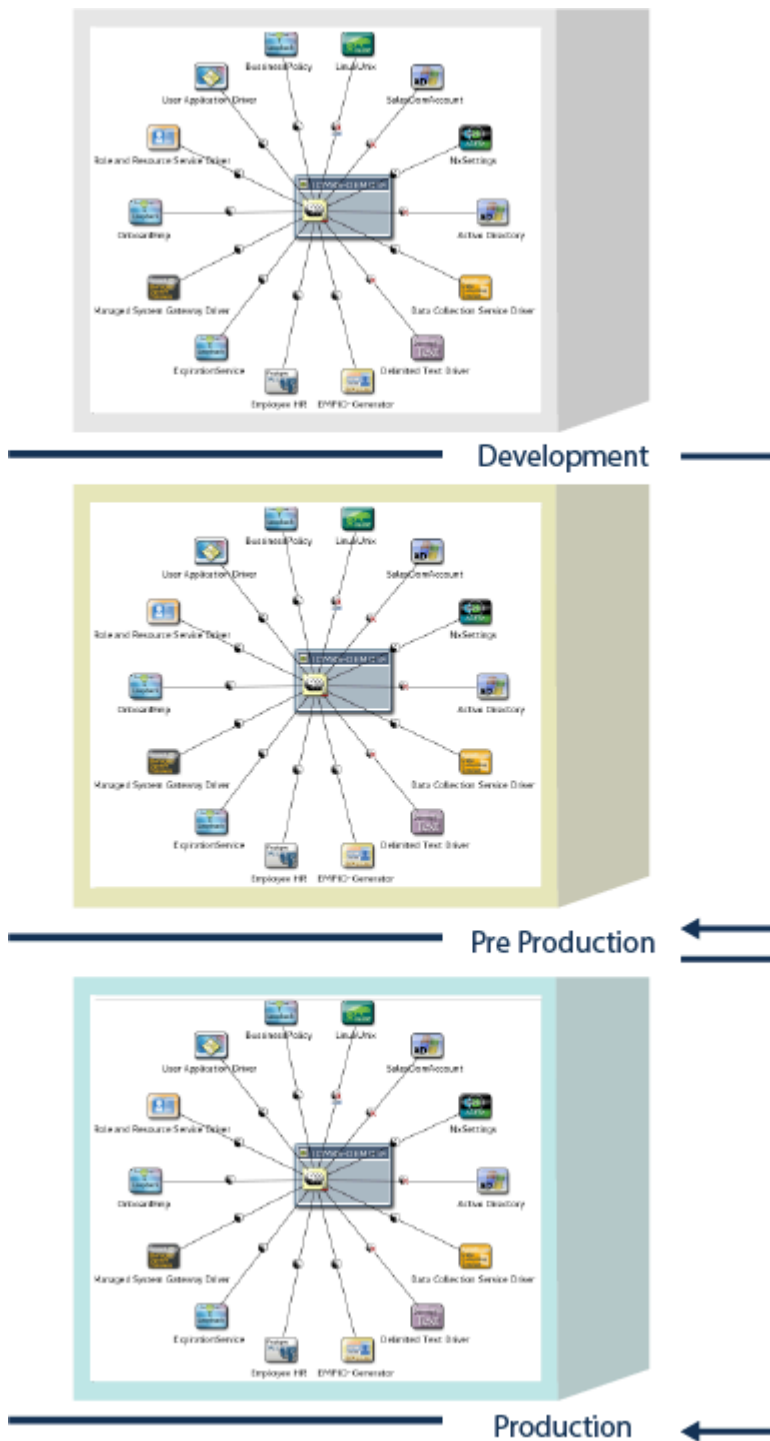
- [Understanding Staging](#)
- [Scope of Staging Documentation](#)
- [Staging Use Cases](#)
- [Using Designer for Staging OpenText Identity Manager Projects](#)
- [Moving Authorizations During Staging](#)

1.1.1. Understanding Staging

Software products must be tested before they are deployed in an IT environment. Typically, after all customizations are completed in the Development stage, the software products are tested, and then moved to the production stage.

To avoid risk to your production environment, it is recommended that you deploy your OpenText Identity Manager projects in separate stages, including the development environment (includes all customizations), the test environment (primarily for testing), and the production environment. This process of developing and testing is called **staging**. To be able to get the right feel of how the product behaves in the actual production environment, ensure that there is uniformity across all stages.

This guide provides step-by-step procedures for staging your Identity Management projects, enabling you to more easily move each project from the initial stage to all subsequent stages. The guide helps with reducing complexity in the product deployment process by helping you to test your OpenText Identity Manager project at multiple stages before the project is live.



Staging OpenText Identity Manager Projects

[Staging OpenText Identity Manager Projects](#) shows a basic representation of the movement of OpenText Identity Manager projects across different stages. The scale of the projects grows with stages, but the scale of an individual driver does not necessarily grow in the same way.

1.1.2. Scope of Staging Documentation

This guide does not discuss the following information:

- All possible aspects of the staging process. The guide primarily focuses on staging the core components of OpenText Identity Manager, particularly the engine and drivers.
- Staging and moving the OpenText Identity Manager User Application or any tools associated with that component.
- Staging Identity Reporting, if installed.

For more information, see the [Administrator's Guide to the Identity Applications](#) and the [Administrator's Guide to Identity Reporting](#).

1.1.3. Staging Use Cases

The staging discussion includes the following scenarios. Note that some steps in the staging process do not apply to all use cases.

- New Deployment:** New drivers and applications are developed during the development stage and are then tested and moved to the test environment. These applications are put together and moved to the production environment.
- Existing Deployment:** You already have development, test, and production environments ready and you want to move a new policy from the development environment to the production environment.

1.1.4. Using Designer for Staging OpenText Identity Manager Projects

Designer is used in the OpenText Identity Manager project development to develop packages, policies, configuration files, and other objects that make up the configuration of a driver. Designer can create the OpenText Identity Manager components required for running an OpenText Identity Manager project and then deploy the project on another OpenText Identity Manager deployment.

You can use Designer to segregate the OpenText Identity Manager environment into separate packages that can be easily moved from one stage to another. You can use package versioning (for upgrades and downgrades) to effectively move your changes from one stage to another.

Before you begin to use this guide, you should be familiar with OpenText Identity Manager and Designer. As you create projects, you should have a uniform Identity Vault design across all the states so that common objects are available. Some objects are moved automatically by Designer, but others should be moved explicitly to make them available in the next stage. For more information, see [Preparing for Staging](#).

1.1.5. Moving Authorizations During Staging

Moving authorizations across stages is a key issue for an Identity Vault security model. OpenText eDirectory authorizations are assigned to individual objects or a collection of objects. These authorizations play an important role in the object security because they determine the permission to access the object to which they have been assigned.

OpenText eDirectory authorizations can be performed through Access Control Lists (ACLs) or Security Equivalences/Exclude Roles. Drivers, jobs, RBEs, and so on should have enough permissions to perform the desired operations. For more information, see [Preparing for Staging](#).

1.2. Preparing for Staging

The information covered in the following sections helps you design your OpenText Identity Manager projects to stage them easily:

- [Understanding Packages](#)
- [Converting Configuration Files to Packages](#)
- [Prerequisites](#)
- [Identity Vault Structure](#)
- [Driver Configuration](#)
- [OpenText Identity Manager Objects](#)
- [Rights](#)

1.2.1. Understanding Packages

A package is a container for components of the OpenText Identity Manager driver content, organized according to the functionality you want to provide to a driver. Packages can contain different types of content that you can move from one environment to another, allowing you to reuse content in multiple places. It is easier to track and commit changes using packages. You can move to newer versions of a package while retaining the older ones. Packages also help you to automate the process of moving content changes from one environment to another.

For information about creating custom packages, see [Developing Custom Packages](#) in the [Designer Administration Guide](#).

1.2.2. Converting Configuration Files to Packages

Before you begin the staging process, convert any configuration files in your OpenText Identity Manager environment into packages. Packages are more portable than configuration files and enable you to move your custom policies and related content independently of the server without overwriting any settings required for a particular stage.

For information about converting configuration files to packages, see [Upgrading the Identity Manager Drivers](#) in the [Install and Upgrade Guide for Linux](#) and [Upgrading the Identity Manager Drivers](#) in the [Install and Upgrade Guide for Windows](#).

1.2.3. Prerequisites

Ensure that the following general prerequisites are met before attempting staging:

- All the stages should have the supported version of the OpenText eDirectory, OpenText Identity Manager, and OpenText Identity Manager drivers.
- Designer 4.0 or later is present.
- All the applications and drivers are fully developed and tested in one stage before moving them to the next stage.
- From your project, gather information about the objects that are not modeled by Designer. For more information, see [Objects That Designer Does Not Model](#).
- Create an LDIF file for all the objects not modeled by Designer. Use Designer to import the additional objects.

It's important to be familiar with the recommended best practices for moving OpenText Identity Manager objects between different stages. For more information about staging best practices, see [Staging Best Practices](#).

1.2.4. Identity Vault Structure

An Identity Vault is a flat OpenText eDirectory tree, which consists of several containers for users, devices, groups, objects, and so on. Objects are stored in different containers for performance reasons.

Make sure that you are familiar with the basic principles of directory design. A uniform directory design simplifies administrative tasks for staging.

1.2.5. Driver Configuration

You must create a common data model to allow drivers to work together.

Even though each driver is unique and uses different policies, all drivers use the same guidelines to make the driver configuration file consistent. For example, all policies and driver configuration files have the same naming conventions and support the same common data module.

For guidelines for developing new drivers, see [Identity Manager Driver Configuration Development Guidelines](#).

- [Using GCVs in Policies](#)
- [Simulation and Staging](#)

Using Global Configuration Values in Policies

Global Configuration Values (GCV) are global configuration values or constants, not global variables. A GCV value cannot be changed at runtime. These are globally accessible to the driver and driver set, but not to the tree or network. These can be consumed by all drivers in a driver set or all policies in a driver. For more information about using GCVs, see [Configuring Global Configuration Objects](#) in the [Designer Administration Guide](#).

Simulation and Staging

The Policy Simulator allows you to test and debug a single policy or a group of policies contained in a policy set or all the policies in a driver or a driver set without implementing the policy in the Identity Vault. It also provides a graphical editor to create the XDS Input documents. You can use these features to test the policies without affecting the production environment or the connected system. This means you can use Designer as the first stage in your deployment process by developing and then testing your policies through simulation.

1.2.6. OpenText Identity Manager Objects

Designer provides the ability to develop OpenText Identity Manager projects even in offline mode. You can easily move your OpenText Identity Manager objects from one environment to another. You can export and import projects into a simple configuration file, which can be stored for future use.

Some OpenText Identity Manager objects are not visible in a Designer project, even though they may be necessary for the product installation. To ensure that you move all necessary objects from one stage to another, you should import any objects not modeled in Designer from OpenText eDirectory into an LDIF container, back up those objects by exporting the LDIF container to an external LDIF file, and then import the LDIF file to an LDIF container in the next stage.

Objects That Designer Models

You can model the following objects in Designer:

Object	Description
Driver Sets	A driver set is a container that holds OpenText Identity Manager drivers. Only one driver set can be active on a server at a time. As a result, all active drivers must be grouped into the same driver set.
Drivers	A driver provides the connection between an application and the Identity Vault. The driver is the connector that enables data synchronization and sharing between systems.
GCVs on Driver set and Drivers	Global configuration values (GCVs) are settings that are similar to driver parameters. GCVs can be specified for an individual driver as well as a driver set. If a driver does not have a GCV, the driver inherits the value for that GCV from the driver set.
Policies	Policies cover DirXMLScript, Entitlement, and XSLT.
Libraries	You need to provide a context if the library is outside the driver set.

Object	Description
Resource Objects	Resource objects are mapping tables, GCV resource, prompts, filter resource, or ECMA scripts. The resource objects allow you enhance the functionality of the drivers.
Provisioning Objects	Workflows, roles, resources, teams, etc.
Notification Templates	Notification templates enable you to customize and send e-mail messages that users receive when triggers occur.
Identity Vault Schema, Application Schema	
Role Based Entitlements	You can synchronize data between connected systems. Entitlements allow you to set up criteria for a person or group that, once met, initiate an event to grant or revoke access to business resources within the connected system.
Named Passwords	
Miscellaneous	Credential Application and Credential Repository

Objects That Designer Does Not Model

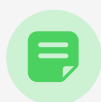
Object	Description
Organization (O), Organizational Unit (OU), Domain (DC), and Container (CN)	Ensure that these objects are created before deploy the project from Designer. Import the containers that contain these objects. Include the following objects in the O, OU, DC, and CN objects: • All objects that are Security Equivalences objects for any drivers. • Objects that are used in any policies. • Objects that are used in any job configurations. • Objects that are used in GCVs.

Object	Description
Users	Ensure that these objects are created before you deploy the project from Designer, especially for the admin users. The list of users can be collected in two different ways: Import the containers that contain the user objects. The following objects must be included in the list: • Security Equivalences and Exclude Administrator Roles for all the drivers. • Static Members on groups and RBE policies. • Search identities and Membership Filter on Dynamic groups and RBE policies. • Users that are used in any policies. • Users that are used in any job configurations. • Users that are used in GCVs.

Object	Description
Groups	Ensure that the static and dynamic group objects are created before deploying them. Import the containers that contain the groups. The following objects must be included in the list: • Groups that are used in any policies. • Groups that are used in any job configurations. • Groups that are used in GCVs.
Password Policies	Ensure that the policies are created before deploying them.
Indices	Ensure that indices are created before deploying them.

Object	Description
Custom Objects	User-defined objects are not defined in Designer. Manually create them before deploying. Import the containers that contain the custom objects. The following objects must be included in the list: • All custom objects that are Security Equivalences objects for all the drivers. • Custom objects that are used in any policies. • Custom objects that are used in any job configurations. • Custom objects that are used in GCVs.

Designer 3.5 and later allow you to import objects listed in the above table in LDIF format and then deploy them along with other objects that are being deployed.



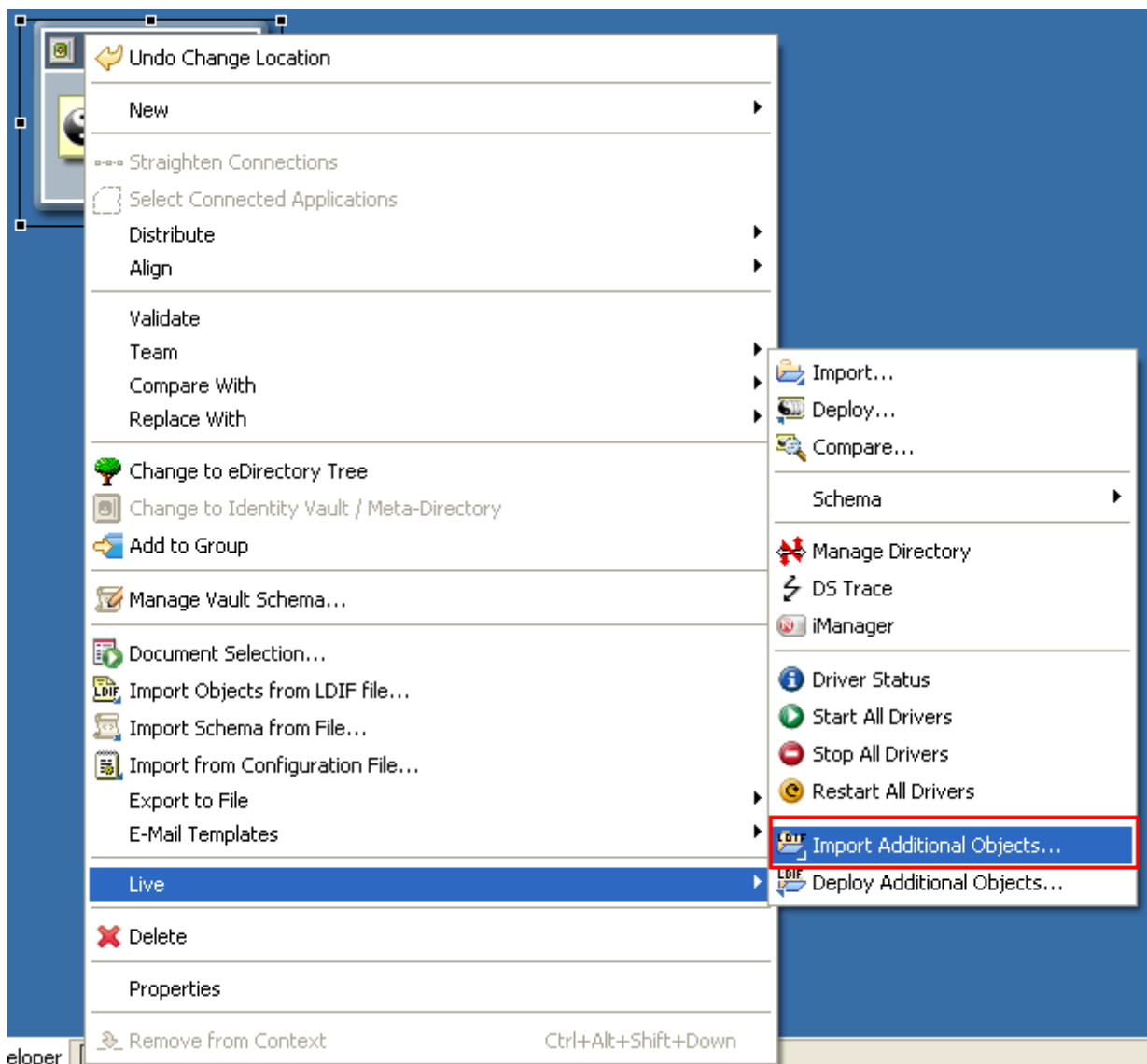
Note

These objects are not modeled as drivers or driver sets in Designer. They can be modified by modifying the LDIF file that contains these objects in Designer. For more information, see [Importing Objects](#).

Importing Objects

Before copying a staged project, you should import any additional objects not modeled in Designer from OpenText eDirectory into an LDIF container. For information about objects not modeled in Designer, see [Objects That Designer Does Not Model](#).

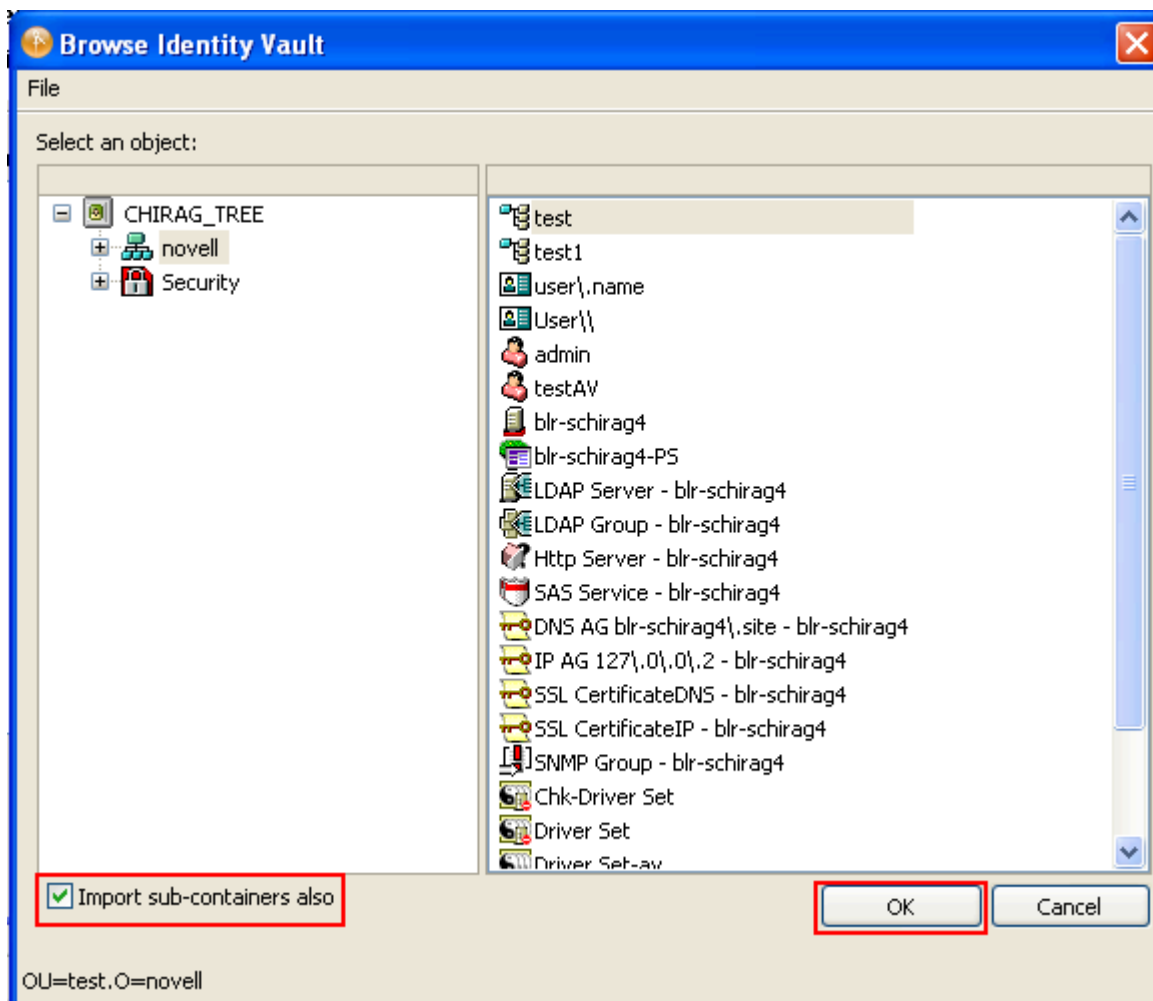
1. In Designer, right-click **Identity Vault** and select **Live > Import Additional Objects**.



2. Browse to and select the objects you want to add to the LDIF file.

Or

If you want to select all the objects in a container, select **Import sub-containers also** in the Browse Identity Vault dialog box.



3. Click **OK**.
4. Click **Continue** on the Import Dialog to import all the objects into Designer. These objects are stored in the LDIF container.



Note

These objects are retrieved by an LDAP channel. If you are running the LDAP service on non-default ports, see [Changing the LDAP Properties](#) for more information.

5. Repeat [Step 1](#) through [Step 4](#) for all the Identity Vaults in your projects.

You can edit the LDIF objects from the LDIF container. Go to the Outline View, expand the Identity Vault, then double-click the LDIF container.

The objects in the LDIF container are overwritten. To keep a record of the objects stored in the LDIF container, export the information of the LDIF container into an LDIF file. For more information about exporting LDIF objects to an LDIF file, see [Exporting LDIF Container Objects to an LDIF File](#).



Important

You should back up your project by using a version control system or export it to a file.

Exporting LDIF Container Objects to an LDIF File

To back up the objects currently stored in the LDIF container in your project, you can export those objects and store them in the LDIF file.

1. In the Modeler, select **Identity Vault > Live > Export to File > Additional Objects**.
2. In the window displayed, specify the file name into which objects stored in the LDIF container should be exported, then click **Save** or **OK**. The following window is displayed in a Windows environment.

If there is no information in the LDIF container, a warning is displayed.

3. If prompted, click **OK**.

Importing Objects from an LDIF File into an LDIF Container

You can import objects from an LDIF file to an LDIF container. This overwrites the existing objects in the LDIF container.

1. In the Modeler, right-click the **Identity Vault** and select **Import Objects from LDIF File**.
2. In the window displayed, select the file from which the LDIF objects should be imported, then click **Open** or **OK**.
3. If prompted, click **OK**.

Deploying Additional Objects into OpenText eDirectory

Designer allows you to update objects that are already present in OpenText eDirectory. However, the current functionality does not support the deployment of forward references. To deploy these objects, you can either manually create the forward references in OpenText eDirectory or remove the references in the LDIF container. For more information about modifying LDIF container data, see [Editing the LDIF Container Data by Using an Editor](#).

Designer allows you to update objects that are in OpenText eDirectory.

1. In the Modeler, select **Identity Vault > Live > Deploy Additional Objects**.
2. Select or deselect the objects by clicking the **Select All** icon, then click **Deploy**.

If the objects are already present in OpenText eDirectory, a warning is displayed.

3. If the objects are already in OpenText eDirectory, click **Update Existing Objects in eDirectory** to update them, click **Deploy**, then click **OK**.

If there are no objects or the information is not in a proper format in the LDIF container, a warning is displayed.

Editing the LDIF Container Data by Using an Editor

Designer allows you to modify the LDIF objects by using different editors. Under the Identity Vault in the Outline view, right-click the LDIF container and select **Open With > Designer Built-in Editor**. You can also double-click the LDIF container to open the container.

You can then use the built-in editor to modify the LDIF data stored in the container.

Deleting the LDIF Container

Designer allows you to delete the LDIF container. Under the Identity Vault in the Outline view, right-click the LDIF container and select **Delete**.

1.2.7. Rights

Driver Equivalences

Designer allows you to define and deploy the Security Equivalences objects for the drivers in OpenText eDirectory.

Security Equivalences require rights to the objects within the Identity Vault to perform tasks on them. For example, an Oracle™ database driver has a policy to create a user in the Identity Vault in a container every time a user is created in the database, but the driver doesn't have enough permissions on the container to create the user, so the process fails. The driver has similar rights to those of the users/objects who have permissions on the container. All the policies should be carefully evaluated to determine what permissions should be given to the drivers.

Designer 3.5 and later can store the Security Equivalences and Exclude Administrative Roles of drivers in the project and assign them to the drivers. Before moving to another staging environment, ensure that you know the Security Equivalences and Exclude Administrative Roles associated with each driver, and these objects are imported as LDIF objects and moved along with other objects before being assigned in the next stage after deployment.

If the Security Equivalences object and the Exclude Administrative Roles object are stored as LDIF objects, Designer ensures that they are created in the next stage before they are assigned.

For more information about Security Equivalence, see [Establishing a Security Equivalent User](#) in the [Security Guide](#).

Roles Based Entitlements Policies

Roles Based Entitlements policies are used by the Entitlements Service driver, which grants entitlements to and revokes entitlements from the users.

An entitlement policy contains the following:

Membership: The list of users assigned to a policy. A user can be dynamically assigned to a policy when he or she meets the criteria for the policy, or the user can be statically (manually) assigned to the policy.

Entitlements: The list of entitlements associated with the policy. Users assigned to the policy receive all of the entitlements associated with the policy. If the user is removed from the policy, he or she loses all entitlements associated with the policy.

You can assign any Identity Vault objects for which you want the entitlement policy to be a trustee. Each member of the policy becomes a trustee of the objects you add.

There are several reasons why you might want to make the policy a trustee of an object:

- One of the policy's entitlements requires the policy's members to have rights to an object.
- You want to use the policy to assign users as trustees of an object even though rights to the object are not required for an entitlement. In this case, you are using the entitlement policy to grant and revoke trustee rights for members of the policy.

These rights are not stored in Designer. You should assign the rights after moving to the next stage.

Jobs

OpenText Identity Manager has a job scheduling utility that schedules events, such as setting the system to disable an account on a specific day, or initiating a workflow to request an extension for a person to access a corporate resource. The Job Manager runs on every OpenText Identity Manager server in the background. Based on the job definition, it checks every minute to see if a job needs to run. When it encounters a job, it runs the appropriate Job implementation.

The Job Manager needs appropriate permissions to run successfully. For example, a job that disables a user account from the Identity Vault needs adequate permissions. Appropriate access must be granted to the job object in the Identity Vault so that it can modify a user object. Use OpenText Identity Console to grant the required rights for the jobs because Designer does not allow you to grant rights for jobs.

1.3. Staging OpenText Identity Manager Projects

- [Staging a Project for the First Time](#)
- [Staging Changes in an Existing Project](#)
- [Copying Java Class .jar Files Between Stages](#)
- [Post-Staging Tasks](#)
- [Changing the LDAP Properties](#)

1.3.1. Staging a Project for the First Time

Ensure that all the applications and OpenText Identity Manager systems are up and running in the next stage before moving the configurations. You can stage projects using packages or configuration files, as appropriate, in your environment.

Staging Using Packages

The simplest, most efficient way to stage your OpenText Identity Manager project is by using the package functionality included in version 4.0 and later.

We recommend using this approach because, unlike configuration files, packages are configured to keep server-specific settings separate from the actual OpenText Identity Manager content. You move all your policies from one stage to the next, not your server configurations.

For more information about converting configuration files to packages, see the following sections:

- [Converting Configuration Files to Packages](#)
- [Upgrading the Identity Manager Drivers](#) in the [Install and Upgrade Guide for Linux](#)
- [Upgrading the Identity Manager Drivers](#) in the [Install and Upgrade Guide for Windows](#)

Staging Using Configuration Files

If you have version 3.6, you may need to perform the staging process using configuration files. Because of the difficulty inherent in updating configuration files, it is not recommend using this process. Instead, convert your existing configuration files to packages.

For more information about converting configuration files to packages, see the following sections:

- [Converting Configuration Files to Packages](#)
- [Upgrading the Identity Manager Drivers](#) in the [Install and Upgrade Guide for Linux](#)
- [Upgrading the Identity Manager Drivers](#) in the [Install and Upgrade Guide for Windows](#)

First-Time Staging Process

To stage an OpenText Identity Manager project for the first time, complete the following steps:

1. Import any additional objects not modeled in Designer from OpenText eDirectory into an LDIF container. For more information on importing additional objects, see [Importing Objects](#).
2. Compare and import any schema changes from the Identity Vault (OpenText eDirectory) to the schema in Designer:
 1. Right-click **ID Vault > Live > Schema > Compare**.
 2. In the Information pane, select **Update Designer**.
 3. Click **Reconcile**.
 4. Click **OK**.
3. (Optional) If you want to keep a backup of your first-stage project, you can export the existing project to an archive file:
 1. Right-click the first-stage project and select **Export Project**.
 2. Select the project you want to export.
 3. Click **Browse** and specify the name of the archive file you want to use, then click **OK**.
 4. Click **Finish**.
 5. Click **OK**.
4. Copy the first-stage project to reuse it in the next stage:
 1. In Designer, go to **Window > Show View > Project**.
 2. Right-click the first-stage project and select **Copy Project**.
 3. Enter the name for the second-stage project. We recommend you use a name that clearly indicates the project is used for the second stage of the staging process.
 4. Click **OK**.
5. (Optional) After copying the existing first-stage project, you may want to rename the project to specify that the project is for the first stage. Complete the following steps to rename the first-stage project:

1. In the project view in Designer, right-click the first stage project and select **Rename**.
2. Specify a new name for the project and click **OK**.
6. In the project view, expand the second-stage project and double-click **System Model**.
7. Change the configuration of one of the Identity Vaults in your project.
 1. In the Outline view or the Modeler, double-click the **ID Vault**.
 2. In the Configuration page, change the **Hostname**, **Admin Username**, and **Admin Password** settings to match those of the Identity Vault you want to use for the second-stage project.
 3. Click **Test Connection** to check the connectivity, then click **OK**.
 4. If necessary, add more servers and associate those servers with the driver set.
8. (Optional) If your second-stage project uses one or more different connected systems, change the configuration of the connected system or systems of the second-stage Identity Vault. To change the system configuration, complete the following steps:
 1. In the Modeler, double-click a driver or a driver line.
 2. In the Driver Configuration page, change the authentication information in the **Authentication** tab.
 3. In the Driver properties page, change the driver-related information in the **Driver Parameters** tab.
 4. The driver parameters depend on the servers on which the drivers reside. Ensure that you change the driver parameters on multiple servers if you have multiple servers running a driver.
9. (Optional) If your second-stage project uses a different connected system or different configuration settings for provisioning, change the GCVs for the drivers and driver sets of the second-stage Identity Vault as necessary.

GCVs should be the only changes you make to the drivers and the driver set along with the configuration. Your policies won't change if they are properly designed.

 1. Update all the GCVs that change with the environment, as necessary.

2. Move or add new GCVs to any new servers added in [Step 7.d](#).

10. To ensure the integrity of your project, run the Project Checker:

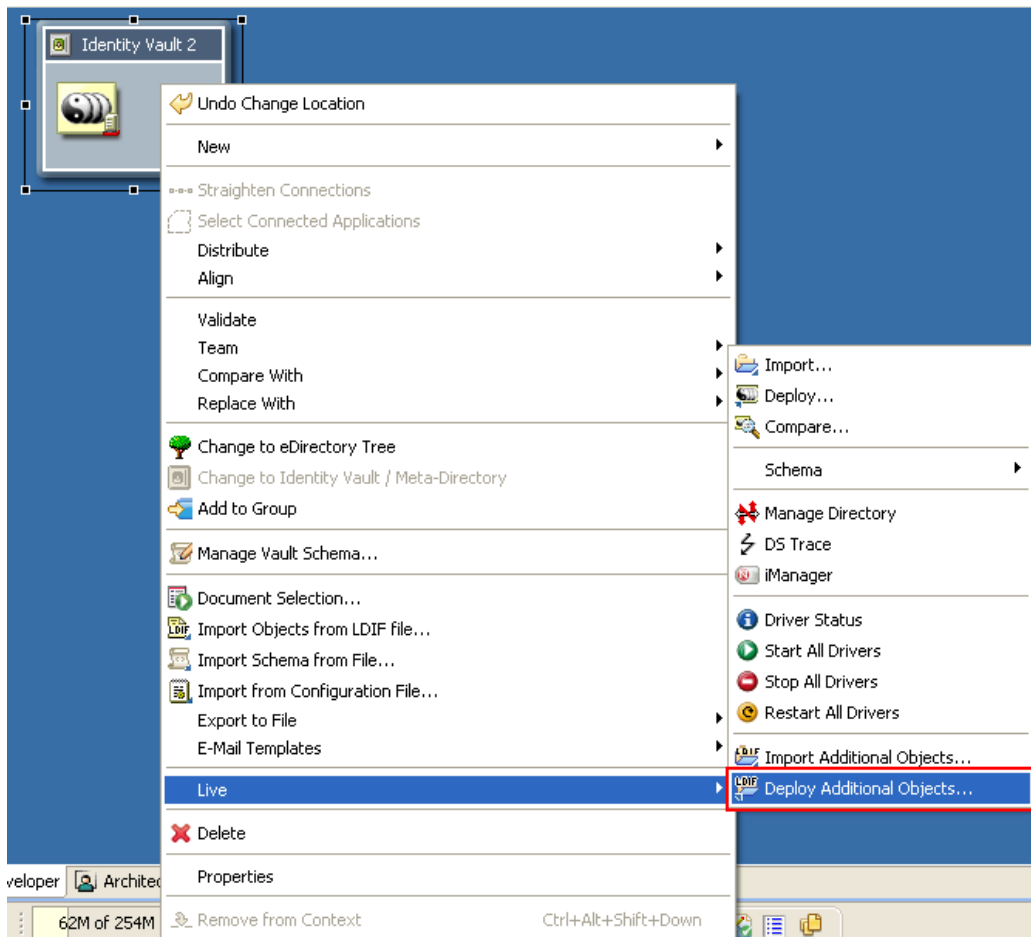
1. Click **Window > Show View > Project Checker**.
2. In the Project Checker view, click the **Run the Project Checker** icon  .
3. Review the results and correct any issues. For more information about using the Project Checker, see [Checking Your Projects](#) in the [Designer Administration Guide](#).

11. Compare and import any schema changes from Designer into the second stage of the Identity Vault:

Compare the schema in Designer with the OpenText eDirectory schema and deploy:

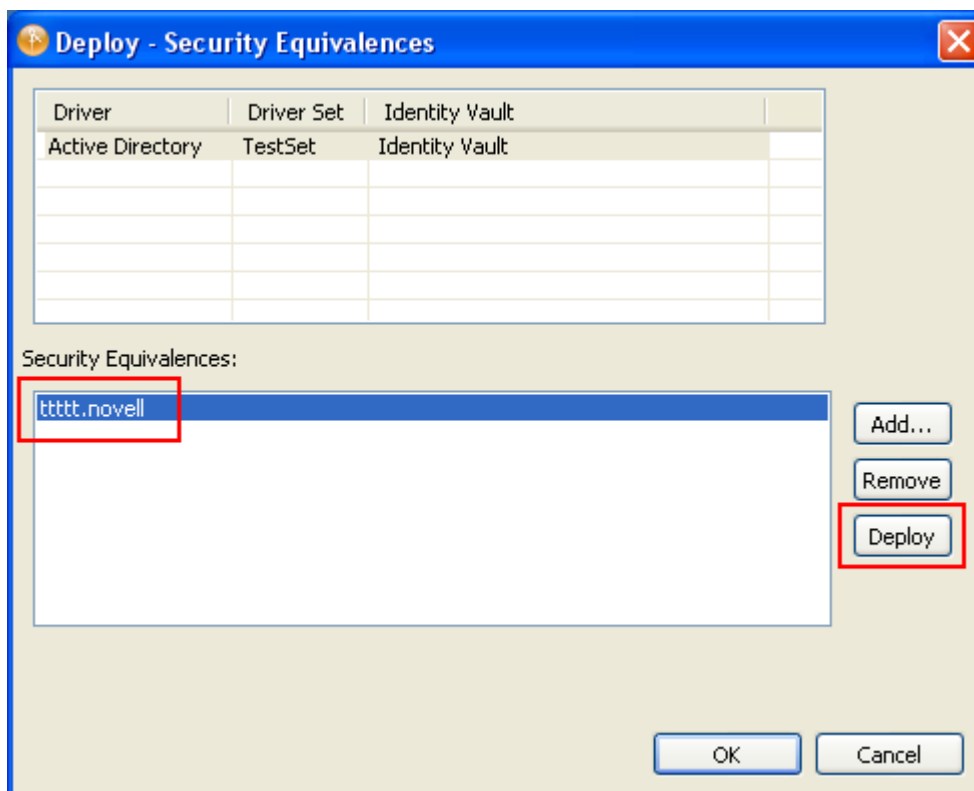
1. Right-click **ID Vault > Live > Schema > Compare**.
2. In the Information pane, select **Update eDirectory**.
3. Click **Reconcile**.
4. Click **OK**.

12. In the Modeler, right-click the Identity Vault and select **Live > Deploy Additional Objects** to deploy additional objects gathered in the [Prerequisites](#).



13. To deploy the Identity Vault, right-click **ID Vault > Live > Deploy**.

14. Deploy the appropriate Security Equivalences and Exclude Admin Roles objects for each driver. See [Rights](#) for more information.



15. Repeat [Step 7](#) through [Step 14](#) for each Identity Vault in your project.

1.3.2. Staging Changes in an Existing Project

After you have staged an OpenText Identity Manager project using packages, you can move the changes from the current stage to the subsequent stages.



Important

It is recommended not to use the configuration files to move changes from one stage to another, because any customizations you have made can be overwritten. Instead, convert your existing configuration files into packages.

For more information about converting configuration files to packages, see [Converting Configuration Files to Packages](#).

1. Right-click the package and select **New Package Version**.
2. In the New Version window, increment the patch-level version number by 1, then click **Next**.
3. Click **Next** to confirm the existing package information, then click **Finish**.
Designer creates a new version of the change package with an incremented version number and a new date stamp.
4. In Designer, make any changes or customizations to your driver to a single package in your first-stage project. You can then test those changes in a non-production environment before moving any customizations to a subsequent stage for production use.
5. If you modify any policies, entitlements, or prompts on the first-stage driver, right-click each object in the Outline view and select **Sync to Package**.



Note

Any changes not synced to the change package will not be moved to the next stage.

6. When finished making changes, navigate to the change package in the Package Catalog.
7. Right-click the new version of the change package and select **Build**.

8. Right-click the package and click **Build Release**.

This releases the version with a timestamp to avoid versioning issues.

9. Click **Browse**, then browse to and select the directory where you want to build the package.

10. Click **OK**.

11. Review the summary information, then click **OK**.

12. In the Project view, open your second-stage project.

13. In the Outline view, right-click **Package Catalog** and select **Import Package**.

14. Click **Browse**, then browse to and select the .jar file for the first-stage package you built, and click **OK**.

15. Click **OK** to import the selected package.

16. Review the import message, then click **OK**.

1.3.3. Copying Java Class .jar Files Between Stages

If you use any custom Java-based functionality in your OpenText Identity Manager policies, you must ensure you move that functionality from one stage to the next. Designer does not automatically move this functionality between stages.

To move this functionality, you must use SCP or FTP to manually copy each related Java class, stored in a .jar file, from the first-stage IDM server to the second-stage IDM server. Note that this process takes place outside of Designer and requires moving the actual files from one computer to another, rather than from one project to another.

OpenText Identity Manager typically stores Java classes in the following directory:

```
/opt/novell/edirectory/lib/dirxml/classes
```

However, in some environments, Java classes may be stored in a different Java classpath. To determine if your driver set uses a different Java classpath, complete the following steps:

1. Right-click the driver set in the Designer Modeler and select **Properties**.
2. Click **Java**. Designer displays any additional Java classpath locations in the **Classpath additions** field.

1.3.4. Post-Staging Tasks

Designer does not move all the configurations to the next stage. Users are expected to manually perform a few tasks to ensure that the configurations work properly.

- **Security Equivalences and Exclude Admin Roles:** Check whether all the drivers have appropriate Security Equivalences and Exclude Admin Roles objects, as defined in the previous stage. For more information, see [Driver Equivalences](#).
- **eDir2eDir Driver Certificates:** If you have eDir2eDir driver certificates created in the current stage, ensure that these certificates are created in the next stage.
 1. In Designer, right-click the **eDir2eDir** driver link and select **Secure Connection Settings**.
 2. Click **Enable SSL/TLS**, select the required options, then click **OK**.
 3. Right-click the **eDir2eDir** driver link, then click **Live > Create eDir-to-eDir Certificates**.
- **Java Environment Parameters:** The Java* environment parameters enable you to configure the Java Virtual Machine™ (JVM) on the OpenText Identity Manager server associated with the driver set. You might need to change the Java classpath options if the .jar files your OpenText Identity Manager server is looking for reside at a different place in the new stage. To change the location, go to **DriverSet > Properties > Java > Classpath additions** and provide the correct classpaths. When you enter multiple classpaths, separate them with a semi colon (;) for a Windows JVM and a colon (:) for a UNIX* or Linux* JVM. Deploy the driver set if you make any changes.
- **Indexes:** Make sure that all the customized indexes from the previous stage have been copied to the new stage. OpenText eDirectory uses these indexes to significantly improve the query performance. Some indexes are shipped with OpenText eDirectory. These default indexes are for the following attributes:
 - CN
 - Aliased Object Name
 - dc
 - Obituary
 - Given Name

- Member
- Surname
- Reference
- uniqueID
- Equivalent to Me
- GUID
- NLS: Common Certificate
- cn_SS
- Revision
- uniqueID_SS
- extensionInfo
- ldapAttributeList
- ldapClassList

You can visit each Identity Vault server and collect the customized index information.

- **Password Policies:** Ensure that password policies assigned to the containers, users, groups in the previous stage are assigned again in the current stage.
- **Challenge Response Objects:** In addition to password policies, ensure that you migrate or recreate any challenge response objects used in the previous stage in the current stage. You can either import your existing challenge response objects into the LDIF container in the first-stage project or note the details of the challenge response objects in the first-stage project and create new objects in the next stage.

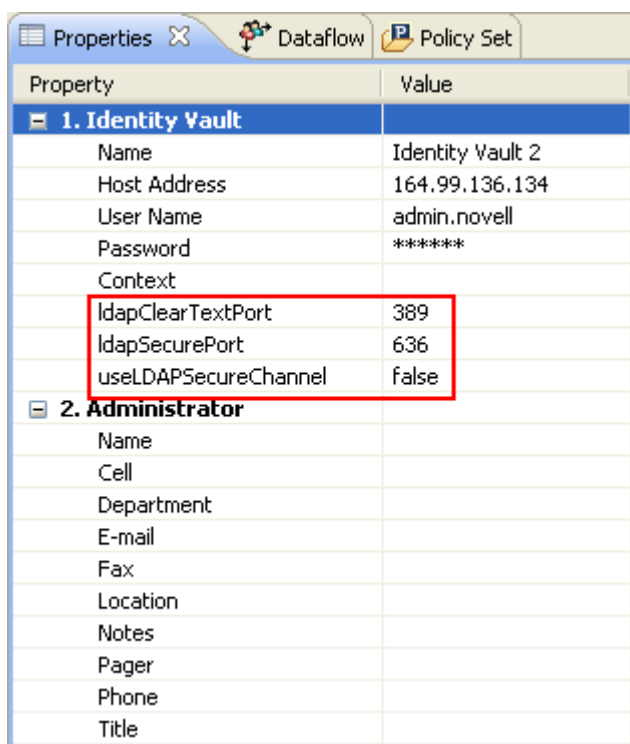
For information importing objects into the LDIF container, see [Importing Objects](#).

- **Restarting All Drivers:** Start the drivers after moving the driver configuration to the next stage. In the Modeler, right-click each driver and select **Driver > Start Driver**.

1.3.5. Changing the LDAP Properties

You can modify non-default LDAP properties by using the Properties view of the Identity Vault. This is used for importing and deploying objects.

1. Under the Properties view, click **ID Vault**, then specify the LDAP clear text port and LDAP secure port numbers.



The screenshot shows the 'Properties' view in Identity Manager. The 'Dataflow' tab is selected. The table lists properties for '1. Identity Vault'. A red box highlights the 'IdapClearTextPort', 'IdapSecurePort', and 'useLDAPSecureChannel' properties.

Property	Value
1. Identity Vault	
Name	Identity Vault 2
Host Address	164.99.136.134
User Name	admin.novell
Password	*****
Context	
IdapClearTextPort	389
IdapSecurePort	636
useLDAPSecureChannel	false
2. Administrator	
Name	
Cell	
Department	
E-mail	
Fax	
Location	
Notes	
Pager	
Phone	
Title	

2. Save the project.

1.4. Staging Best Practices

- If you delete drivers and driver sets from Stage 2 in order to deploy the drivers from Stage 1, you can lose the associations.
- We recommend you use the same Designer workspace for all stage projects.
- Don't deploy Stage 1 objects directly into the Stage 2 environment.
- When performing the staging process, ensure you store server-specific settings as GCVs. If using packages, you can then leave those GCVs behind when you change stages. However, if using configuration files, you need to copy the GCVs from one stage to the next so they do not get overwritten.
- When creating new GCVs for staging, ensure you add those GCVs at the driver set level.
- Before moving to any stage, understand the existing stage and the objects that Designer does not automatically bring in (see [Objects That Designer Does Not Model](#)) for the next stage.

Ensure that you know which objects are required in the subsequent stages. Consolidate these objects in the LDIF file.

- You can store any OpenText eDirectory objects not modeled in Designer as a DS object in your first-stage project and add that object to a package so that you can move the DS object to the second-stage.
- Ensure that you assign the Security Equivalences, Trustees, and Server Certificates of Stage 1 in Stage 2 after deployment.
- LDIF files that contain additional objects should be stored locally. You can use the Import Convert Export (ICE) utility to deploy these objects in any stage.
- For a new deployment in Stage 2, ensure that LDIF objects are deployed before importing the configuration file or the project file.
- For an existing deployment in Stage 2, ensure that you compare the existing project with the Stage 1 configuration, deploy the necessary LDIF objects, then import the configuration file.
- Ensure that objects are up-to-date when you import them into the LDIF file.

Always import the additional objects into Stage 1 before moving to Stage 2.

-
- Export the additional objects of Stage 1 into an LDIF file before moving to Stage 2 so that these objects can be manually created in Stage 2 before deployment.
 - Rather than directly modifying your filters, we recommend you create a filter object in your first-stage project, add that filter object to a package, and then install the package on a driver in your first-stage project. Using the package, you can then easily move any changes to the filter to the next stage.



© Copyright 2026 Open Text

For more info, visit <https://docs.microfocus.com>
